目录

一 LINUX 平台下的基本命令 **2**

1 文件和目录操作 2

2 文件内容查看与处理命令 3

3 系统信息查看命令 5

4 权限相关命令 7

5 管道符组合命令 8

二 LINUX 下的 shell 编程案例 **11**

1 程序功能说明 11

2 Shell 脚本执行方法 11

3 案例展示 12

4 源代码展示 12

5 Shell 程序调试方法 17

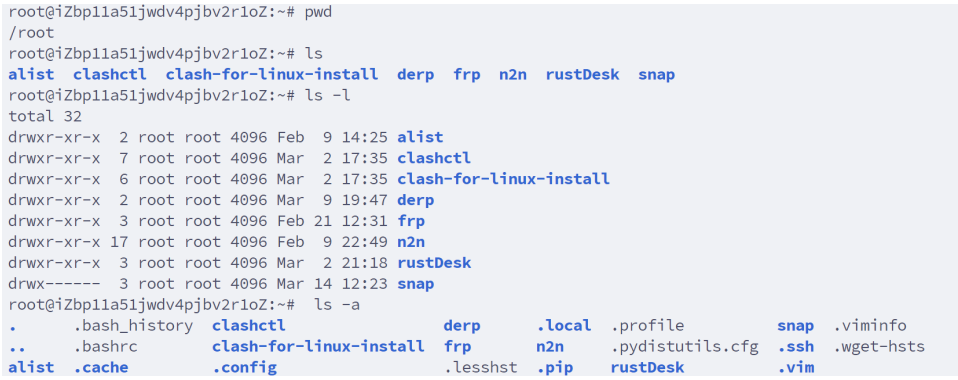
一 LINUX 平台下的基本命令

作业中要求使用 Linux 平台进行操作，我正好在阿里云租了一台 2 核 2g 的服务器，安装的系统是 Ubuntu 24.04。这个作业我将会使用 ssh 连接到我的服务器，并在服务器上进行操作。我使用 termius 这个工具来连接服务器。

我会分类别来操作一些基础命令。

1 文件和目录操作

1. 查看当前所在目录：`pwd`
2. 查看当前目录下的文件：`ls`
3. 以详细形式查看文件：`ls -l`
4. 查看隐藏文件：`ls -a`



```
root@iZbp11a51jwdv4pjbv2r1oZ:~# pwd
/root
root@iZbp11a51jwdv4pjbv2r1oZ:~# ls
alist  clashctl  clash-for-linux-install  derp  frp  n2n  rustDesk  snap
root@iZbp11a51jwdv4pjbv2r1oZ:~# ls -l
total 32
drwxr-xr-x  2 root root 4096 Feb  9 14:25 alist
drwxr-xr-x  7 root root 4096 Mar  2 17:35 clashctl
drwxr-xr-x  6 root root 4096 Mar  2 17:35 clash-for-linux-install
drwxr-xr-x  2 root root 4096 Mar  9 19:47 derp
drwxr-xr-x  3 root root 4096 Feb 21 12:31 frp
drwxr-xr-x 17 root root 4096 Feb  9 22:49 n2n
drwxr-xr-x  3 root root 4096 Mar  2 21:18 rustDesk
drwx------  3 root root 4096 Mar 14 12:23 snap
root@iZbp11a51jwdv4pjbv2r1oZ:~#  ls -a
.      .bash_history  clashctl                derp      .local  .profile          snap  .viminfo
..     .bashrc        clash-for-linux-install frp       n2n     .pydistutils.cfg  .ssh  .wget-hsts
alist  .cache         .config                 .lesshst  .pip    rustDesk          .vim
```

图 1 命令 1.1-1.4

5. 创建一个目录：`mkdir linux_test`
6. 进入目录：`cd linux_test`
7. 创建空文件：`touch file1.txt`
8. 向文件写入内容：`echo "hello linux" > file1.txt`
9. 查看文件内容：`cat file1.txt`
10. 复制文件：`cp file1.txt file2.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~# mkdir linux_test
root@iZbp11a51jwdv4pjbv2r1oZ:~# cd linux_test
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# touch file1.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# ehco "hello, linux" > file1.txt
Command 'ehco' not found, did you mean:
  command 'ehlo' from snap ehlo (0.0.83)
  command 'echo' from deb coreutils (9.4-3ubuntu6.2)
See 'snap info <snapname>' for additional versions.
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# echo "hello, linux" > file1.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# cat file1.txt
hello, linux
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# cp file1.txt file2.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# cat file2.txt
hello, linux
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test#
```

图 2 命令 1.5-1.10

11. 重命名文件：`mv file1.txt newfile.txt`
12. 删除文件：`rm newfile.txt`
13. 返回上一级目录：`cd ..`
14. 删除目录：`rm -r linux_test`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# mv file1.txt  newfile.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# ls
file2.txt  newfile.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# rm newfile.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# ls
file2.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_test# cd ..
root@iZbp11a51jwdv4pjbv2r1oZ:~# ls
alist  clashctl  clash-for-linux-install  derp  frp  linux_test  n2n  rustDesk  snap
root@iZbp11a51jwdv4pjbv2r1oZ:~# rm -r linux_test/
root@iZbp11a51jwdv4pjbv2r1oZ:~# ls
alist  clashctl  clash-for-linux-install  derp  frp  n2n  rustDesk  snap
root@iZbp11a51jwdv4pjbv2r1oZ:~#
```

图 3 命令 1.11-1.14

2 文件内容查看与处理命令

我先使用如下命令创建了一个测试文件夹，然后测试命令

```
mkdir linux_homework
cd linux_homework\
echo -e "apple\nbanana\napple\norange\nbanana\nlinux\ncommand\npipe" > words.txt
```

1. 查看完整内容：`cat words.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~# mkdir linux_homework
root@iZbp11a51jwdv4pjbv2r1oZ:~# ls
alist  clashctl  clash-for-linux-install  derp  frp  linux_homework  n2n  rustDesk  snap
root@iZbp11a51jwdv4pjbv2r1oZ:~# cd linux_homework/
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# echo -e "apple\nbanana\napple\norange\nbanana\nlinux\nco
mmand\npipe" > words.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# ls
words.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# cat words.txt
apple
banana
apple
orange
banana
linux
command
pipe
```

图 4 命令 2.1

2. 查看文件前 3 行：`head -n 3 words.txt`
3. 查看文件后 3 行：`tail -n 3 words.txt`
4. 统计文件行数、单词数、字符数：`wc words.txt`
5. 统计文件行数：`wc -l words.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# head -n 3 words.txt
apple
banana
apple
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# tail -n 3 words.txt
linux
command
pipe
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# wc words.txt
 8  8 52 words.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# wc -l words.txt
8 words.txt
```

图 5 命令 2.2-2.5

6. 对文件内容排序：`sort words.txt`
7. 去除相邻重复行：`uniq words.txt`
8. 查找包含 apple 的行：`grep "apple" words.txt`
9. 查找包含 banana 的行，并显示行号：`grep -n "banana" words.txt`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# sort words.txt
apple
apple
banana
banana
command
linux
orange
pipe
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# uniq words.txt
apple
banana
apple
orange
banana
linux
command
pipe
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# grep "apple" words.txt
apple
apple
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# grep -n "banana" words.txt
2:banana
5:banana
```

图 6 命令 2.6-2.9

3 系统信息查看命令

1. 查看系统内核和系统信息：`uname -a`
2. 查看当前用户名：`whoami`
3. 查看当前日期时间：`date`
4. 显示日历：`cal`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# uname -a
Linux iZbp11a51jwdv4pjbv2r1oZ 6.8.0-90-generic #91-Ubuntu SMP PREEMPT_DYNAMIC Tue Nov 18 14:14:30 UTC 2
025 x86_64 x86_64 x86_64 GNU/Linux
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# whoami
root
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# date
Sat Apr 25 03:57:12 PM CST 2026
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# cal
     April 2026
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30
```

图 7 命令 3.1-3.4

其中第四个命令需要先安装 ncal，我使用 `sudo apt install ncal` 安装之后就可以正常使用了。

5. 查看磁盘空间：`df -h`
6. 查看内存使用情况：`free -h`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           162M  1.5M  160M   1% /run
efivarfs        256K  7.4K  244K   3% /sys/firmware/efi/efivars
/dev/vda3        40G  5.8G   32G  16% /
tmpfs           807M     0  807M   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
/dev/vda2       197M  6.2M  191M   4% /boot/efi
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/f6fdeb7a0935974bbc5df63262aeeeb1
eab665b7deb467c5473618aaa7edeb16
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/237c5d820903ac7889c5633f264a81a5
cfbd5beb6d68c2e52780dcf004e0fbc2
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/8b23e10ea750512823dc7b98534178b2
485c5248054f51408fb1b2651dcba3a4
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/27db7f8296651a194aea4a077a1dfe1e
2ca5ca9a74782f6885fae7792df73f73
tmpfs           162M   12K  162M   1% /run/user/0
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# free -h
               total        used        free      shared  buff/cache   available
Mem:           1.6Gi       664Mi       226Mi       3.3Mi       889Mi       948Mi
Swap:             0B          0B          0B
```

图 8 命令 3.5-3.6

7. 查看系统进程和资源占用：`top`

```
Myserver
top - 16:00:00 up 75 days, 23:44,  2 users,  load average: 0.01, 0.02, 0.00
Tasks: 141 total,   1 running, 140 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  4.8 sy,  0.0 ni, 95.2 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   1613.0 total,    207.5 free,    688.7 used,    884.0 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    924.3 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   22940  13056   8576 S   0.0   0.8  15:38.32 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:01.70 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      9 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     12 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
     13 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     16 root      20   0       0      0      0 S   0.0   0.0   0:27.08 ksoftirqd/0
     17 root      20   0       0      0      0 I   0.0   0.0   3:25.52 rcu_preempt
     18 root      rt   0       0      0      0 S   0.0   0.0   0:18.02 migration/0
     19 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     21 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     22 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/1
     23 root      rt   0       0      0      0 S   0.0   0.0   0:16.23 migration/1
     24 root      20   0       0      0      0 S   0.0   0.0   0:21.53 ksoftirqd/1
     26 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
```

图 9 命令 3.7

8. 查看当前系统进程：`ps aux`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.7  22940 13056 ?        Ss   Feb08  15:38 /usr/lib/systemd/systemd --system --
root           2  0.0  0.0      0     0 ?        S    Feb08   0:01 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    Feb08   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-rcu_g]
root           5  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-rcu_p]
root           6  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-slub_]
root           7  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-netns]
root           9  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/0:0H-events_highpri]
root          12  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-mm_pe]
root          13  0.0  0.0      0     0 ?        I    Feb08   0:00 [rcu_tasks_kthread]
root          14  0.0  0.0      0     0 ?        I    Feb08   0:00 [rcu_tasks_rude_kthread]
root          15  0.0  0.0      0     0 ?        I    Feb08   0:00 [rcu_tasks_trace_kthread]
root          16  0.0  0.0      0     0 ?        S    Feb08   0:27 [ksoftirqd/0]
root          17  0.0  0.0      0     0 ?        I    Feb08   3:25 [rcu_preempt]
root          18  0.0  0.0      0     0 ?        S    Feb08   0:18 [migration/0]
root          19  0.0  0.0      0     0 ?        S    Feb08   0:00 [idle_inject/0]
root          20  0.0  0.0      0     0 ?        S    Feb08   0:00 [cpuhp/0]
root          21  0.0  0.0      0     0 ?        S    Feb08   0:00 [cpuhp/1]
root          22  0.0  0.0      0     0 ?        S    Feb08   0:00 [idle_inject/1]
root          23  0.0  0.0      0     0 ?        S    Feb08   0:16 [migration/1]
root          24  0.0  0.0      0     0 ?        S    Feb08   0:21 [ksoftirqd/1]
root          26  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/1:0H-events_highpri]
root          27  0.0  0.0      0     0 ?        S    Feb08   0:00 [kdevtmpfs]
root          28  0.0  0.0      0     0 ?        I<   Feb08   0:00 [kworker/R-inet_]
root          29  0.0  0.0      0     0 ?        S    Feb08   0:02 [kauditd]
root          31  0.0  0.0      0     0 ?        S    Feb08   0:04 [khungtaskd]
root          33  0.0  0.0      0     0 ?        S    Feb08   0:00 [oom_reaper]
```

图 10 命令 3.8

由于使用的是我的服务器，所以系统进程比较多。没有全部截下来。

9. 查看历史命令：`history`

```
 801  uname -a
 802  whoami
 803  date
 804  cal
 805  apt install ncal
 806  cal
 807  clear
 808  uname -a
 809  whoami
 810  date
 811  cal
 812  clea
 813  clear
 814  df -h
 815  top
 816  df -h
 817  clear
 818  df -h
 819  clear
 820  top
 821  clear
 822  free -h
 823  df -h
 824  free -h
 825  clear
 826  ps aux
 827  clear
 828  history
```

图 11 命令 3.9

历史记录中包含了我之前使用过的命令，包括一些测试命令和安装命令等。这里只截了本次课程展示的几个历史命令。

4 权限相关命令

1. 创建脚本文件：`touch test.sh`

2. 查看文件权限：`ls -l test.sh`
3. 给文件添加可执行权限：`chmod +x test.sh`
4. 再次查看权限变化：`ls -l test.sh`
5. 查看文件详细状态信息：`stat test.sh`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# touch test.sh
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# ls -l test.sh
-rw-r--r-- 1 root root 0 Apr 25 16:07 test.sh
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# chmod +x test.sh
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# ls -l test.sh
-rwxr-xr-x 1 root root 0 Apr 25 16:07 test.sh
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# stat test.sh
  File: test.sh
  Size: 0               Blocks: 0          IO Block: 4096   regular empty file
Device: 253,3   Inode: 1179125     Links: 1
Access: (0755/-rwxr-xr-x)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-04-25 16:07:12.238643435 +0800
Modify: 2026-04-25 16:07:12.238643435 +0800
Change: 2026-04-25 16:07:26.812080642 +0800
 Birth: 2026-04-25 16:07:12.238643435 +0800
```

图 12 命令 4.1-4.5

5 管道符组合命令

管道符 | 的作用是将前一个命令的输出结果作为后一个命令的输入。通过管道符，可以把多个简单命令组合成一个更复杂的数据处理流程。例如 `cat words.txt | sort | uniq -c | sort -nr` 先读取文件内容，再进行排序，然后统计重复行出现次数，最后按照出现次数从高到低排序，体现了 Linux 命令组合使用的灵活性。

1. 把文件内容传给 sort 排序后去重：`cat words.txt | sort | uniq`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# cat words.txt | sort | uniq
apple
banana
command
linux
orange
pipe
```

图 13 命令 5.1

2. 统计每个单词出现次数：`cat words.txt | sort | uniq -c`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# cat words.txt | sort | uniq -c
      2 apple
      2 banana
      1 command
      1 linux
      1 orange
      1 pipe
```

图 14 命令 5.2

3. 找出包含字母 a 的行，并排序：`grep "a" words.txt | sort`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# cat words.txt | grep "a" | sort
apple
apple
banana
banana
command
orange
```

图 15 命令 5.3

4. 在当前目录中筛选.txt 文件：`ls | grep ".txt"`
5. 在进程列表中查找 bash 相关进程：`ps aux | grep bash`
6. 查看最近使用的命令：`history | tail`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# ls -l | grep ".txt"
-rw-r--r-- 1 root root 52 Apr 25 15:47 words.txt
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# ps aux | grep bash
root      219565  0.0  0.0   2616   128 ?        Ss   Mar09   0:00 /bin/sh -c bash /app/build_cert.sh $
DERP_HOST $DERP_CERTS /app/san.conf &&     /app/derper --hostname=$DERP_HOST     --certmode=manual
--certdir=$DERP_CERTS     --stun=$DERP_STUN      --a=$DERP_ADDR     --http-port=$DERP_HTTP_PORT     --v
erify-clients=$DERP_VERIFY_CLIENTS
root      243312  0.0  0.2   9052  4480 tty1     S+   Mar14   0:00 -bash
root      568015  0.0  0.3   9300  6016 pts/0    Ss   15:46   0:00 -bash
root      568468  0.0  0.1   6544  2304 pts/0    S+   16:12   0:00 grep --color=auto bash
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# history | tail
  833  ls -l test.sh
  834  stat test.sh
  835  cat words.txt | sort
  836  cat words.txt | sort | uniq
  837  cat words.txt | sort | uniq -c
  838  cat words.txt | uniq -c
  839  cat words.txt | grep "a" | sort
  840  ls -l | grep ".txt"
  841  ps aux | grep bash
  842  history | tail
```

图 16 命令 5.4-5.6

7. 筛选磁盘挂载信息：`df -h | grep "/dev"`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# df -h | grep "/"
tmpfs           162M  1.5M  160M   1% /run
efivarfs        256K  7.4K  244K   3% /sys/firmware/efi/efivars
/dev/vda3        40G  5.8G   32G  16% /
tmpfs           807M     0  807M   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
/dev/vda2       197M  6.2M  191M   4% /boot/efi
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/f6fdeb7a0935974bbc5df63262aeeeb1
eab665b7deb467c5473618aaa7edeb16
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/237c5d820903ac7889c5633f264a81a5
cfbd5beb6d68c2e52780dcf004e0fbc2
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/8b23e10ea750512823dc7b98534178b2
485c5248054f51408fb1b2651dcba3a4
overlay          40G  5.8G   32G  16% /var/lib/docker/rootfs/overlayfs/27db7f8296651a194aea4a077a1dfe1e
2ca5ca9a74782f6885fae7792df73f73
tmpfs           162M   12K  162M   1% /run/user/0
```

图 17 命令 5.7

8. 统计文件行数：`cat words.txt | wc -l`
9. 统计单词出现次数，并按出现次数从高到低排序：`cat words.txt | sort | uniq -c | sort -nr`

```
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# cat words.txt | wc -l
8
root@iZbp11a51jwdv4pjbv2r1oZ:~/linux_homework# cat words.txt | sort | uniq -c | sort -nr
      2 banana
      2 apple
      1 pipe
      1 orange
      1 linux
      1 command
```

图 18 命令 5.8-5.9

二 LINUX 下的 shell 编程案例

本次作业我设计并实现了一个基于 Bash Shell 的 Linux 命令答题系统。程序通过菜单提供开始答题、查看历史成绩、清空历史成绩和退出系统等功能，题目从外部题库文件中读取，答题结束后会把本次成绩保存到成绩文件中。这个案例既能练习 Linux 基础命令，也能体现 Shell 脚本中条件判断、循环控制、函数封装、文件读写和程序调试等知识点。

1 程序功能说明

本程序的主要功能如下：

- 显示主菜单，用户可以通过输入编号选择不同功能。
- 从 `questions.txt` 题库文件中逐行读取题目和标准答案。
- 判断用户输入是否为空、是否与标准答案一致，并根据结果计算得分和错误次数。
- 当答错次数达到设定上限时，系统自动结束本轮答题。
- 将每次答题的时间、得分、已答题数和错误题数保存到 `score.txt` 文件。
- 支持查看历史成绩，并使用 `awk` 统计历史最高分。
- 支持清空历史成绩，清空前需要用户再次确认，避免误操作。

程序采用“主菜单 + 功能函数”的结构，整体逻辑比较清晰。用户进入程序后，主循环会不断显示菜单，直到用户选择退出。每个功能被封装成独立函数，例如 `init_env`、`show_menu`、`start_quiz`、`save_score`、`show_history` 和 `clear_history`，这样可以减少重复代码。

2 Shell 脚本执行方法

在 Linux 平台下，可以进入脚本所在目录后执行以下命令运行程序：

```
cd code/task2
chmod +x linux_quiz.sh
./linux_quiz.sh
```

其中，`chmod +x linux_quiz.sh` 用于给脚本添加可执行权限，`./linux_quiz.sh` 表示执行当前目录下的脚本文件。

3 案例展示

接上次的作业，这次这个也是在我的 Ubuntu 服务器上运行和测试这个 shell 文件，之后的调试也是在这台服务器上进行。

首先创建一个目录来存放本次作业。然后加入 sh 文件和 txt 题库文件。

```
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/  clashctl/  clash-for-linux-install/  derp/  frp/  linux_homework/  n2n/  rustDesk/  snap/
root@iZbp11a51jwdv4pjbv2r1oZ ~# mkdir task2
root@iZbp11a51jwdv4pjbv2r1oZ ~# ls
alist/     clash-for-linux-install/  frp/             n2n/       snap/
clashctl/  derp/                     linux_homework/  rustDesk/  task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~# cd task2/
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# touch linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim linux_quiz.sh
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# touch questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# vim questions.txt
root@iZbp11a51jwdv4pjbv2r1oZ ~/task2# ls
linux_quiz.sh  questions.txt
```

4 源代码展示

以下是我的本次 shell 编程作业的完整源代码：

```bash
#!/usr/bin/env bash

QUESTION_FILE="./questions.txt"
SCORE_FILE="./score.txt"
WRONG_LIMIT=2

# 初始化运行环境
init_env() {
    if [ ! -f "$SCORE_FILE" ]; then
        touch "$SCORE_FILE"
    fi
}

# 暂停，便于用户查看结果
pause_screen() {
    echo
    read -r -p " 按 Enter 键返回主菜单..."
}

# 显示主菜单
show_menu() {
```

```
    echo
    echo "======================================"
    echo "        Linux 命令答题系统"
    echo "======================================"
    echo "1. 开始答题"
    echo "2. 查看历史成绩"
    echo "3. 清空历史成绩"
    echo "0. 退出系统"
    echo "======================================"
    echo
    echo "PS: 答错 $WRONG_LIMIT 道题后系统会自动结束"
    echo
}

# 核心答题功能
start_quiz() {
    score=0
    wrong=0
    total=0
    answered=0

    if [ ! -f "$QUESTION_FILE" ]; then
        echo "题库文件不存在: $QUESTION_FILE"
        return
    fi

    # 使用文件描述符 3 读取题库，避免用户输入被题库文件提前消费。
    while IFS='|' read -r question correct_answer <&3; do
        # 跳过空行
        if [ -z "$question" ]; then
            continue
        fi

        total=$((total + 1))

        echo "-------------------------------------"
        echo "第 $total 题: $question"
        read -r -p " 请输入答案: " user_answer
```

```

        # 去掉用户输入前后的空格
        user_answer="$(echo "$user_answer" | xargs)"

        if [ -z "$user_answer" ]; then
            echo " 答案不能为空，本题判定为错误。"
            wrong=$((wrong + 1))
        elif [ "$user_answer" = "$correct_answer" ]; then
            echo " 回答正确，+1 分。"
            score=$((score + 1))
            answered=$((answered + 1))
        else
            echo "回答错误！正确答案是：$correct_answer"
            wrong=$((wrong + 1))
            answered=$((answered + 1))
        fi

        echo "当前得分：$score，错误次数：$wrong/$WRONG_LIMIT"
        echo

        if [ "$wrong" -ge "$WRONG_LIMIT" ]; then
            echo
            echo "错误次数达到 $WRONG_LIMIT 次，答题结束。"
            break
        fi

        read -r -p " 按 Enter 继续下一题..."
        clear

    done 3< "$QUESTION_FILE"

    echo
    echo "========== 答题结果 ==========="
    echo "本次得分：$score"
    echo "已答题数：$answered"
    echo "错误题数：$wrong"

    save_score "$score" "$answered" "$wrong"
```

```
}

# 保存成绩
save_score() {
    current_score="$1"
    answered_count="$2"
    wrong_count="$3"
    current_time="$(date '+%Y-%m-%d %H:%M:%S')"

    echo "$current_time|$current_score|$answered_count|$wrong_count" >>
    ↪ "$SCORE_FILE"

    if [ -w "$SCORE_FILE" ]; then
        echo "成绩已保存到 $SCORE_FILE"
    else
        echo " 成绩文件不可写，保存失败。"
    fi
}

# 查看历史成绩，并在最上方显示最高分
show_history() {
    echo
    echo "========== 历史成绩 ==========="

    if [ ! -f "$SCORE_FILE" ]; then
        echo " 暂无成绩文件。"
        return
    fi

    if [ ! -s "$SCORE_FILE" ]; then
        echo " 暂无历史成绩。"
        return
    fi

    highest_score="$(awk -F'|' 'BEGIN {max=0} {if ($2 > max) max=$2} END
    ↪ {print max}' "$SCORE_FILE")"

    echo "最高分: $highest_score"
```

```
    echo "-----------------------------------"
    echo " 时间 | 得分 | 已答题数 | 错误题数"
    echo "-----------------------------------"

    while IFS='|' read -r time score answered wrong; do
        if [ -n "$time" ]; then
            echo "$time | $score | $answered | $wrong"
        fi
    done < "$SCORE_FILE"
}

# 清空历史成绩
clear_history() {
    echo
    read -r -p " 确认清空历史成绩吗？输入 y 确认： " confirm

    if [ "$confirm" = "y" ] || [ "$confirm" = "Y" ]; then
        > "$SCORE_FILE"
        echo " 历史成绩已清空。"
    else
        echo " 已取消清空操作。"
    fi
}

# 主程序入口
init_env

while true; do
    clear
    show_menu
    read -r -p " 请输入功能编号： " choice

    case "$choice" in
        1)
            clear
            start_quiz
            pause_screen
            ;;
```

```
        2)
            clear
            show_history
            pause_screen
            ;;
        3)
            clear_history
            pause_screen
            ;;
        0)
            echo " 系统已退出。"
            exit 0
            ;;
        *)
            echo " 输入错误，请输入 0 到 3 之间的数字。"
            ;;
    esac
done
```

5 Shell 程序调试方法

在编写和测试这个脚本时，可以使用以下方法进行调试：

1. 使用 `bash -n linux_quiz.sh` 检查脚本语法错误。该命令只检查语法，不真正执行脚本。
2. 使用 `bash -x linux_quiz.sh` 跟踪脚本执行过程。执行时每条命令都会在终端显示出来，便于观察变量值和分支执行情况。
3. 在关键位置加入 `echo` 输出，例如输出当前得分、错误次数、读取到的题目和答案，用于确认程序状态是否正确。
4. 分模块调试各个函数，例如先测试成绩文件是否能正常创建，再测试答题逻辑，最后测试历史成绩显示和清空功能。
5. 构造异常情况进行测试，例如删除 `questions.txt`、清空 `score.txt`、输入空答案、输入错误菜单编号等，检查程序是否能给出合理提示。

例如，可以用下面的命令进行语法检查和执行跟踪：

```
bash -n linux_quiz.sh
```

```
bash -x linux_quiz.sh
```

通过调试可以发现，程序对题库文件不存在、成绩文件为空、用户输入错误等情况都做了处理，因此整体运行比较稳定。